Week 8

Machine Learning and Big Data - DATA622

CUNY School of Professional Studies

Week 8

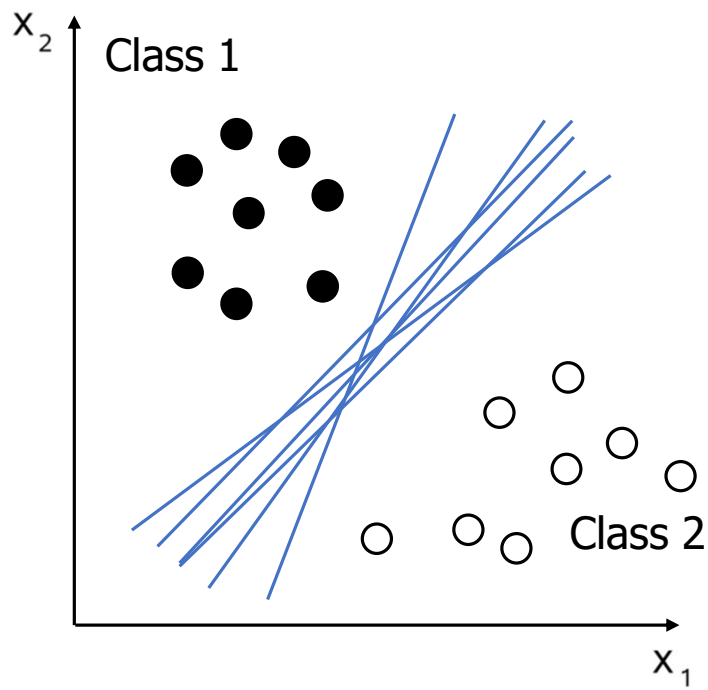
1. Discussion Board Week 8: Support Vector Machines
2. Reading materials:
   - **Textbook Reading: Practical Machine Learning in R (PMLiR)**
   - **ISLR Chapter 9 (Pages 367-396): Support Vector Machines (45 mins)**
     "An Introduction to Statistical Learning" isn't our main textbook but is the definitive source for this topic (see below how to download free copy from author's site). Our standard text doesn't cover SVM.
   - **ML Concepts Reading & Videos**
     - Read: Support Vector Machines: https://cran.r-project.org/web/packages/e1071/vignettes/svmdoc.pdf
     - Read: Support Vector Machines in R: Journal of Statistical Software: https://eric.univ-lyon2.fr/~ricco/tanagra/fichiers/en_Tanagra_SVM_R_Python.pdf
     - Read: SVM Light (Cornell): https://www.cs.cornell.edu/people/tj/svm_light
     - Read: SVM (Scikit Learn.org): SVM (Scikit Learn.org)
     - Video: Maximal Margin Classifier: https://youtu.be/m59UOo5jAFU (12 mins)
     - Video: When you cannot separate the data well: https://youtu.be/_TR28WOymkE (8 mins)
     - Video: Let's use feature expansion: https://youtu.be/ooYwHNvH-YU (16 mins)

Support Vector Machines

CUNY School of Professional Studies

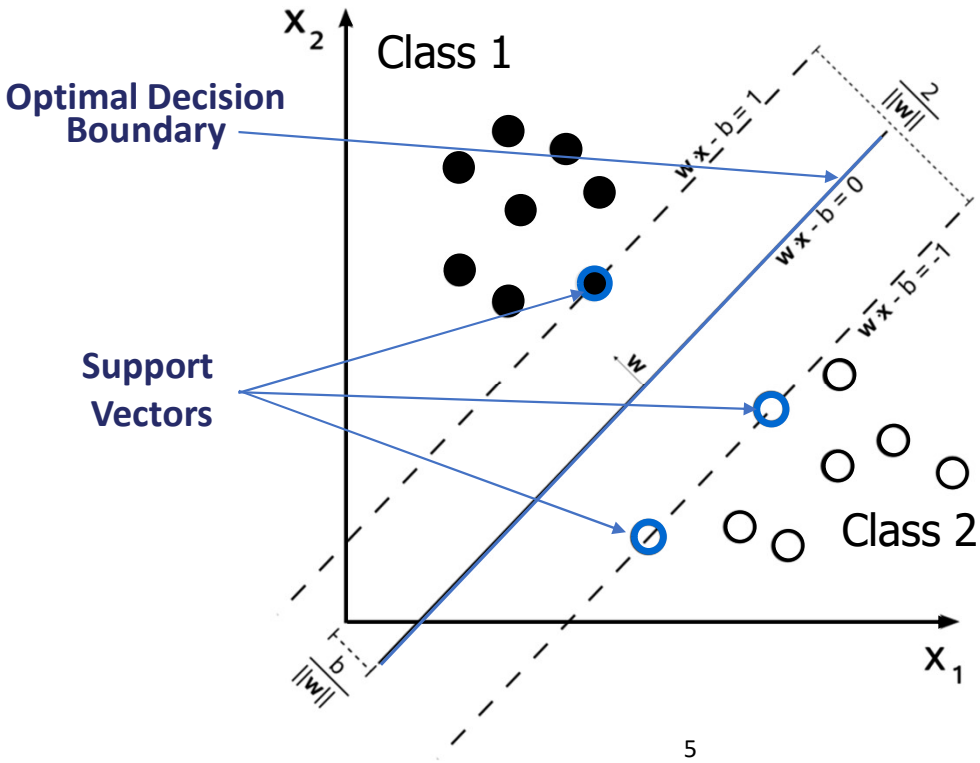
Support Vector Machines

How do you find the right decision boundary?

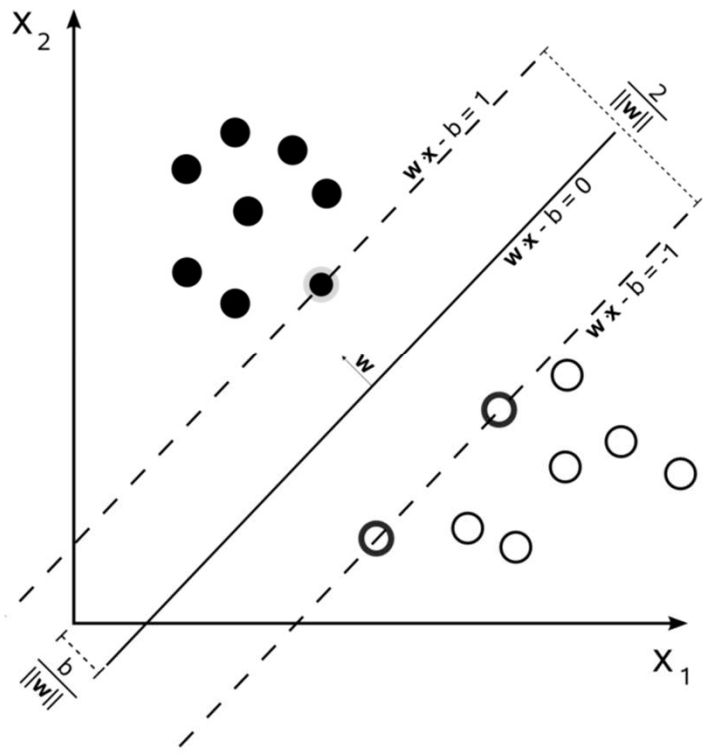


Support Vector Machines

SVM approach: Maximize the margin, to define the decision boundary



Support Vector Machines



$$\max \frac{2}{\|w\|}$$

subject to

$$y_n \begin{Bmatrix} +1 & w^T x + b \geq 1 \\ -1 & w^T x + b \leq -1 \end{Bmatrix}$$

which we can write as $y_n(w^T x + b) \geq 1$

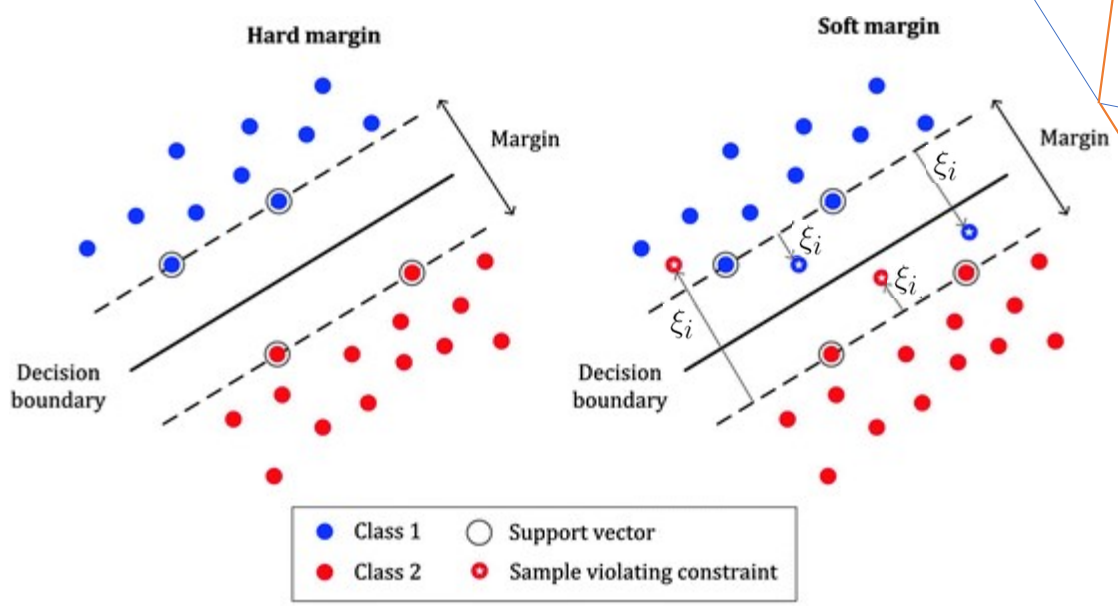
$$max\frac{2}{||w||} = max\frac{1}{||w||} \quad = min||w|| = min\frac{1}{2}||w||^2$$

$$b = \frac{1}{S}\sum_{i=1}^{S}(y_i - w \cdot x)$$

$$w = \sum_{i=1}^{m} \alpha_i y_i x_i$$

Soft-margin SVM

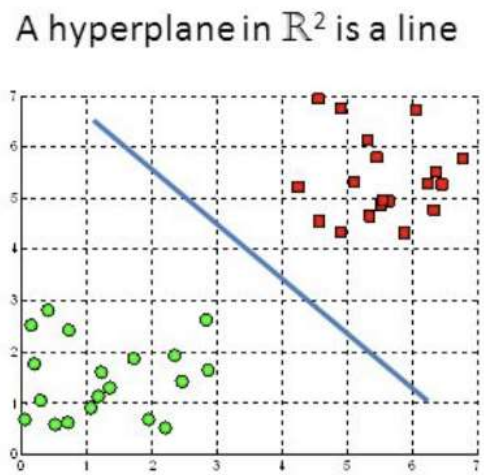
So far we have considered the case with no errors (hard margin).
How do we handle the case where there is is overlap?

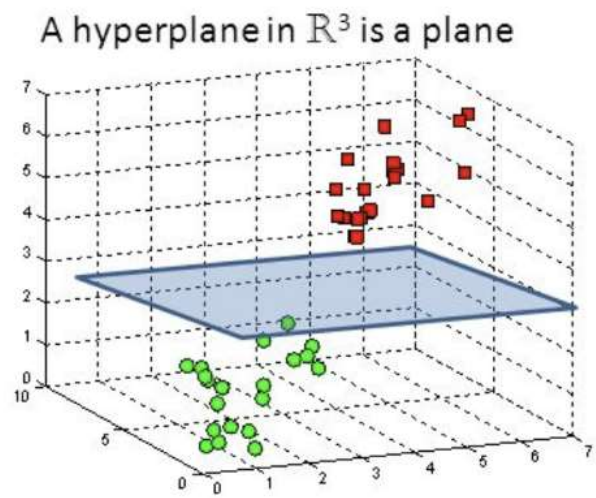


$$\underbrace{\min \frac{1}{2}\|w\|^2}_{\text{regularizer}} + \underbrace{C_i \sum_{i=1}^{n} \xi_i}_{\text{error term}}$$

Hyperplanes

A decision boundary is defined by a hyperplane





CUNY School of Professional Studies

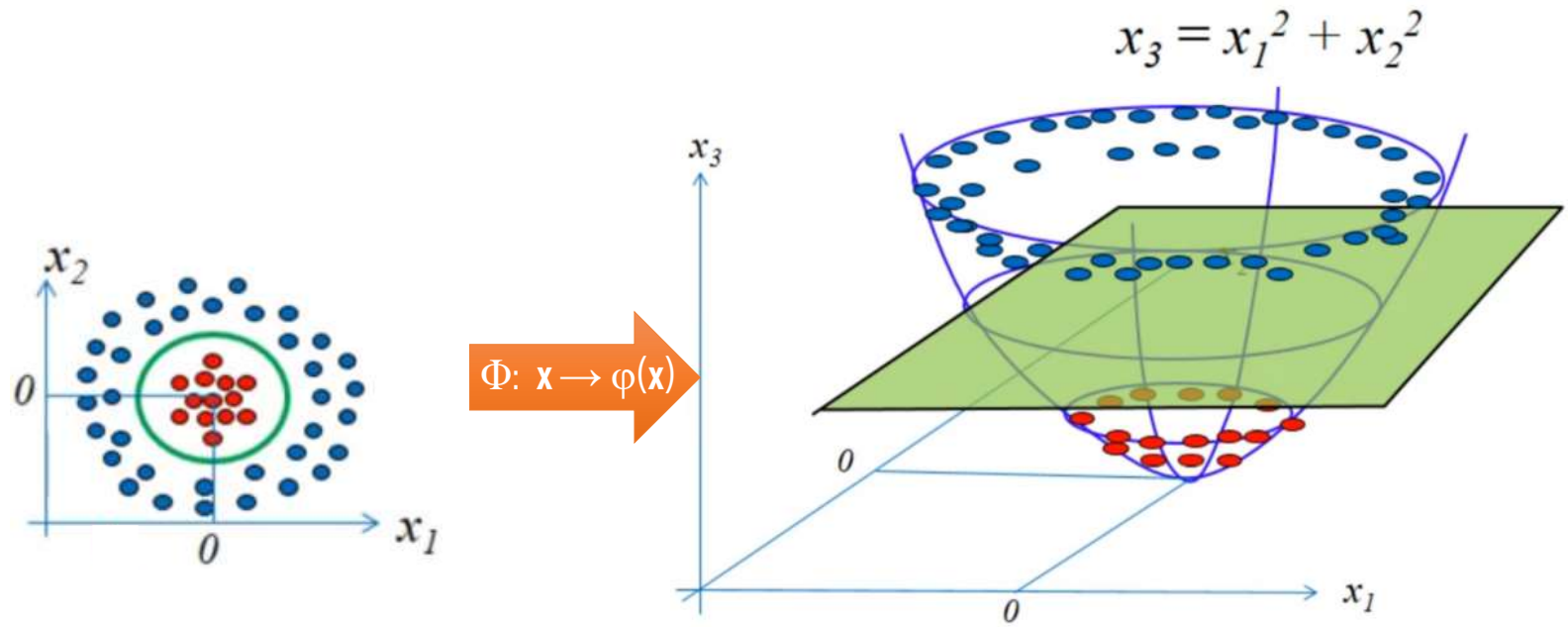
Kernels

What do we do about non-linear decision boundaries?

CUNY School of Professional Studies

Non-linear decision boundaries

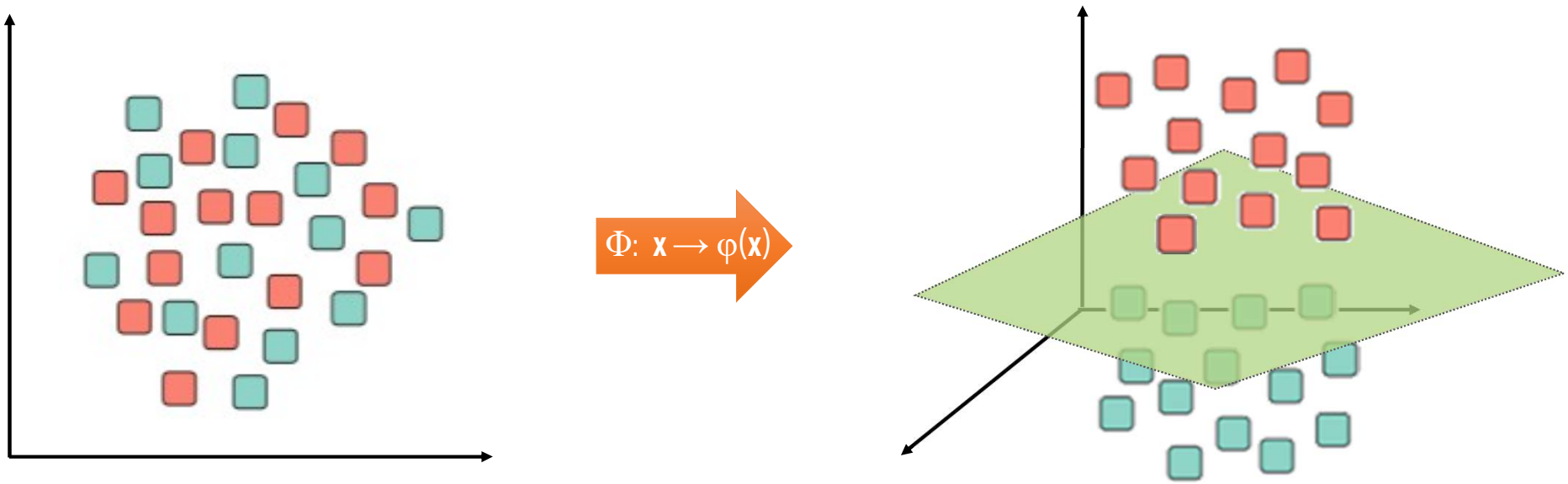
- Map the original input space to a higher-dimensional feature space where the training set is separable.



CUNY School of Professional Studies

The “kernel trick”

- Kernels allow us to keep using a linear classifier – because we have embedded the data in a higher-dimensional feature space



The "kernel trick'

- The linear classifier relies on dot product between vectors:
  $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$
- If every data point is mapped into high-dimensional space via some transformation
  $\Phi: \mathbf{x}_i \rightarrow \boldsymbol{\varphi}(\mathbf{x}_i)$
  the dot product becomes:
  $K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\varphi}(\mathbf{x}_i) \cdot \boldsymbol{\varphi}(\mathbf{x}_j)$
- A *kernel function* is similarity function that corresponds to an inner product in some expanded feature space
- *The kernel trick*: instead of explicitly computing the lifting transformation $\boldsymbol{\varphi}(\mathbf{x})$, define a kernel function K such that:
  $K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\varphi}(\mathbf{x}_i) \cdot \boldsymbol{\varphi}(\mathbf{x}_j)$
- Goal: Avoid having to directly construct $\boldsymbol{\varphi}()$ at any point in the algorithm

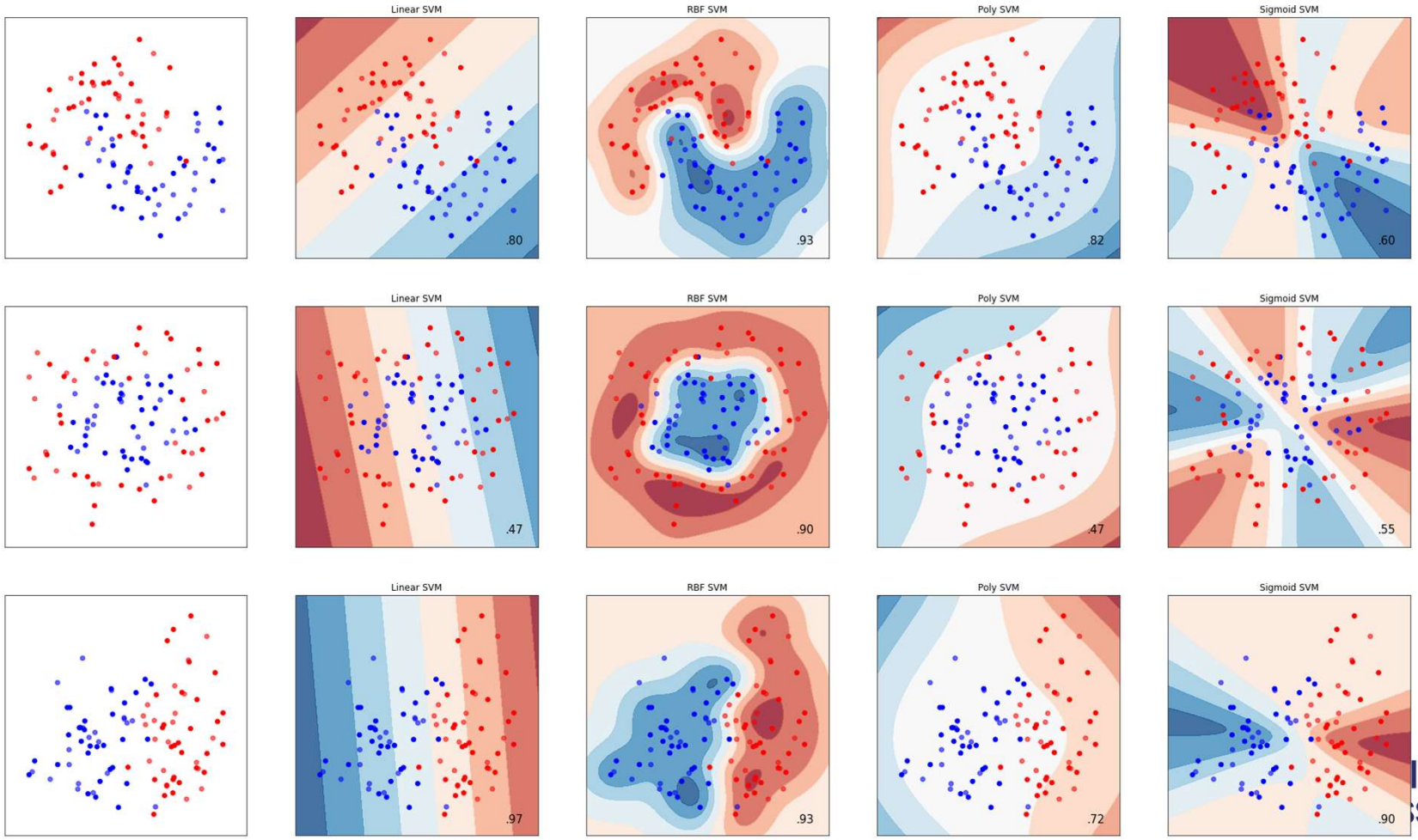
The “kernel trick’

- Kernel trick is allows extremely complex $\phi(\ )$ while keeping $K(a,b)$ simple
- Goal: Avoid having to directly construct $\phi(\ )$ at any point in the algorithm

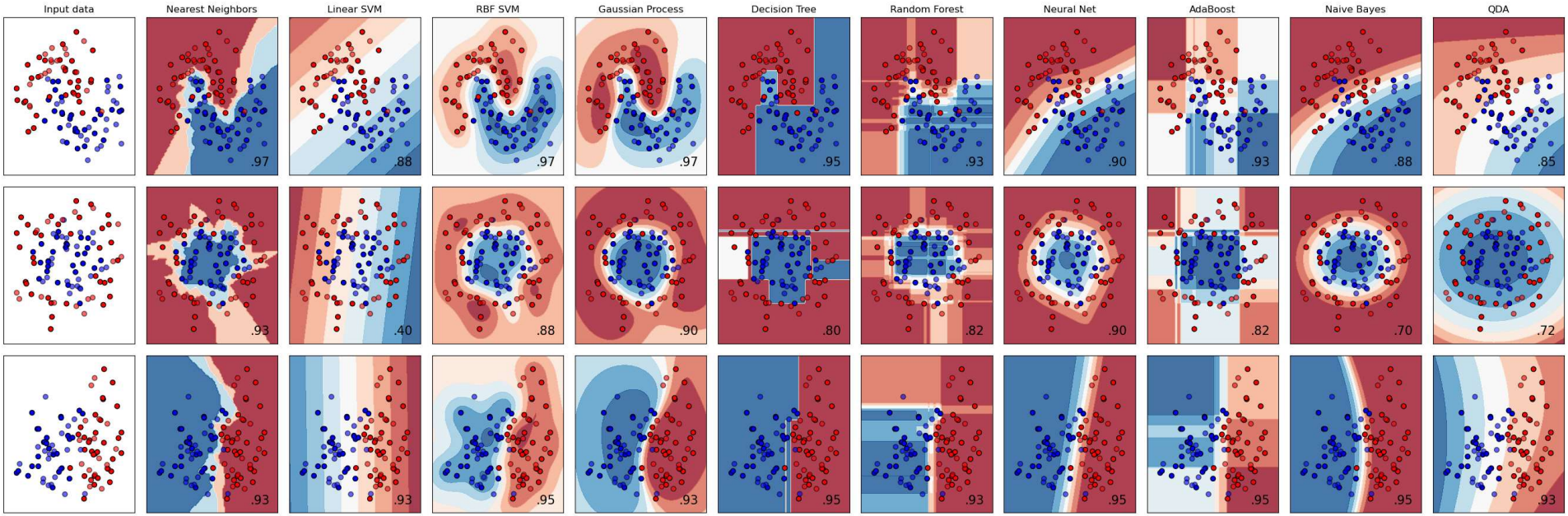
Common kernels

1. Polynomial kernel: $k(\mathbf{x_i}, \mathbf{x_j}) = (\mathbf{x_i} \cdot \mathbf{x_j} + 1)^d$
2. Gaussian kernel: $k(x, y) = \exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$
3. Gaussian radial basis function (RBF): $k(\mathbf{x_i}, \mathbf{x_j}) = \exp(-\gamma\|\mathbf{x_i} - \mathbf{x_j}\|^2)$
4. Laplace RBF kernel: $k(x, y) = \exp\left(-\frac{\|x-y\|}{\sigma}\right)$
5. Sigmoid kernel: $k(x, y) = \frac{J_{v+1}(\sigma\|x-y\|)}{\|x-y\|^{-n(v+1)}}$
6. ANOVA radial basis kernel: $k(x, y) = \sum_{k=1}^{n} \exp(-\sigma(x^k - y^k)^2)^d$
7. Linear splines kernel in one-dimension: $k(x, y) = 1 + xy + xy\ min(x, y) - \frac{x+y}{2}\ min(x, y)^2 + \frac{1}{3}\min(x, y)^3$

Comparison of kernels



Comparison of kernels



Source: https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html

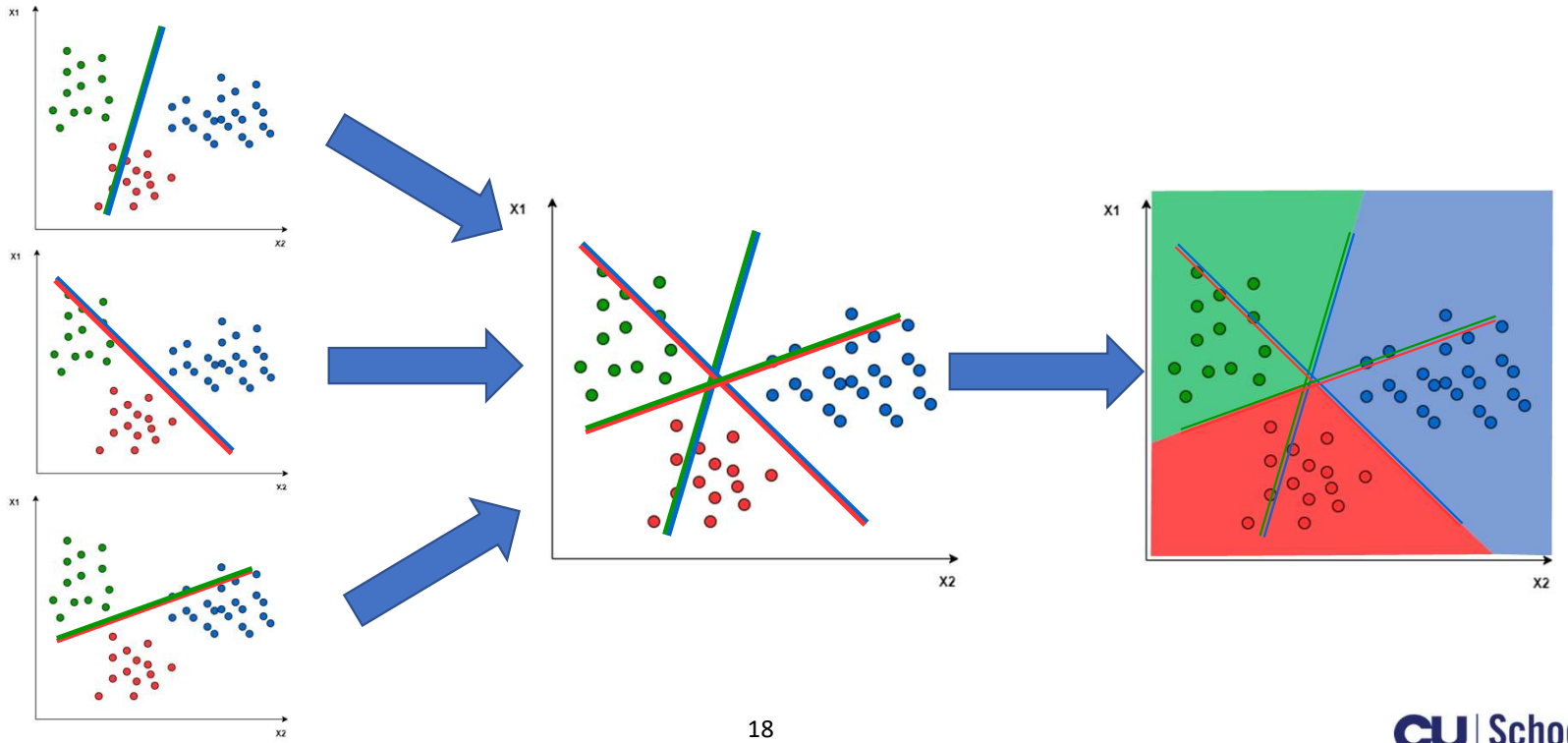
Multi-class Classification

Turning a binary classifier into a multi-class classifier

CUNY School of Professional Studies

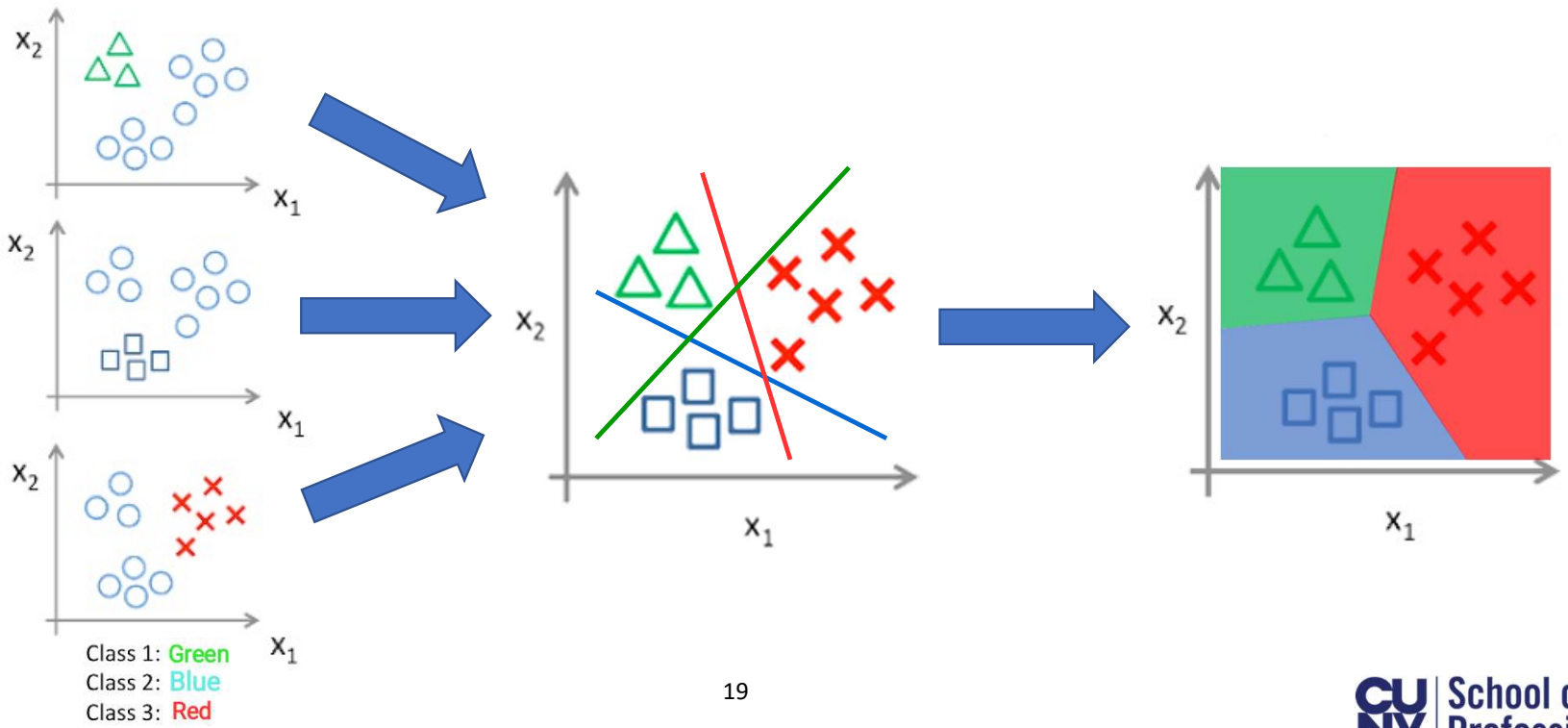
Multi-class SVM: One-vs-one

A hyperplane is generated between every two classes, neglecting the points of the third class.



Multi-class SVM: One-vs-rest

A hyperplane is generated each class and all other classes at once.



Properties of SVMs

Benefits and weaknesses of SVMs

CUNY School of Professional Studies

Benefits of SVMs

- Works well on smaller cleaner datasets
- It can be more efficient because it uses a subset of training points
- Different kernel functions can be specified for the decision function
- Sparseness of solution when dealing with large data sets
  - Only support vectors are used to specify the separating hyperplane
- Ability to handle large feature spaces
  - Complexity does not depend on the dimensionality of the feature space
- Overfitting can be controlled by soft margin approach
- Guaranteed to converge to a single global solution (convex optimization)

Weakness of SVMs

- Sensitive to noise
  - A relatively small number of mislabeled examples can dramatically decrease the performance
- Essentially a binary classifier
- Isn't suited to larger datasets as the training time with SVMs can be high
- Less effective on noisier datasets with overlapping classes
- If the number of features is a lot bigger than the number of data points, avoiding over-fitting when choosing kernel functions and regularization term is crucial.